



Computer Science

06. 교착상태

6주차: Deadlock

? 오늘의 핵심 질문

? 질문 1

프로그램이 실행 중이고 무한루프도 아닌데 왜 아무것도 진행되지 않을까?

? 질문 2

Lock을 많이 걸면 프로그램이 더 안전해질까?

? 질문 3

Deadlock을 미리 막을 수 있을까?

목차 INDEX

01 Deadlock

- Deadlock
- 자원

02 Deadlock의 발생 조건

- Coffman Conditions
- Mutual Exclusion
- Hold and Wait
- 비선점
- Circular Wait

03 Deadlock 대응 전략

- Prevention
- Avoidance
- Detection
- Recovery

04 기타

- Starvation
- Livelock
- Ostrich Algorithm

01

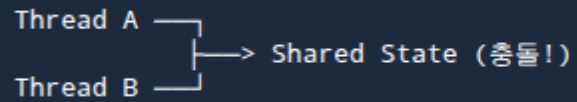
교착 상태

Deadlock

Review: Race Condition & Lock

⚡ 5주차 배운 내용

여러 Thread가 동시 접근 시 공유 데이터가 꼬이는 **Race Condition** 발생



Thread A
Thread B

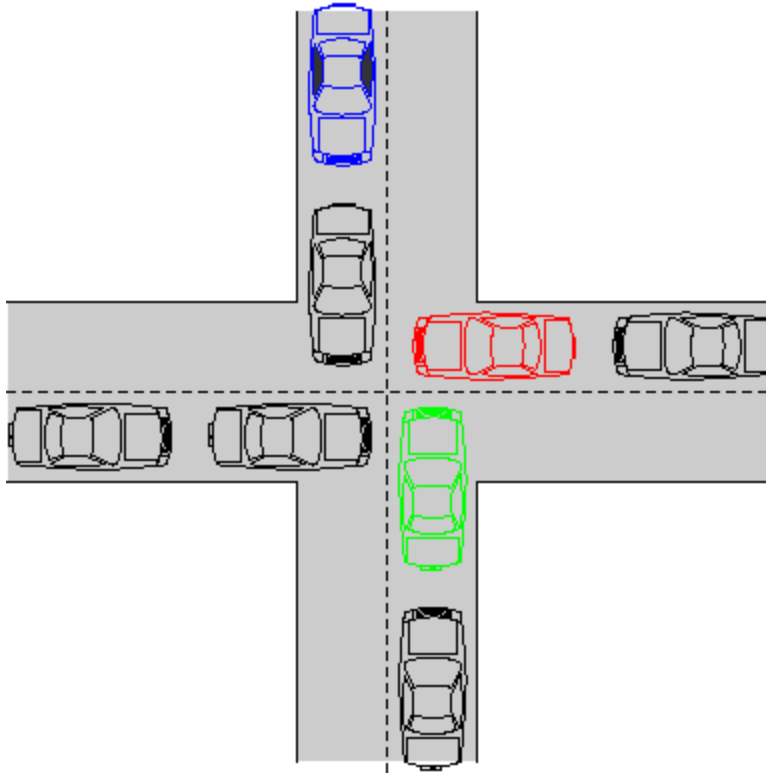
→ Shared State (충돌!)

🔑 해결책: Lock 사용

임계 영역(Critical Section)에 한 번에 하나만 진입하도록
Mutual Exclusion 보장

[Lock 획득] ➡ Critical Section ➡ [Lock 해제]

Deadlock

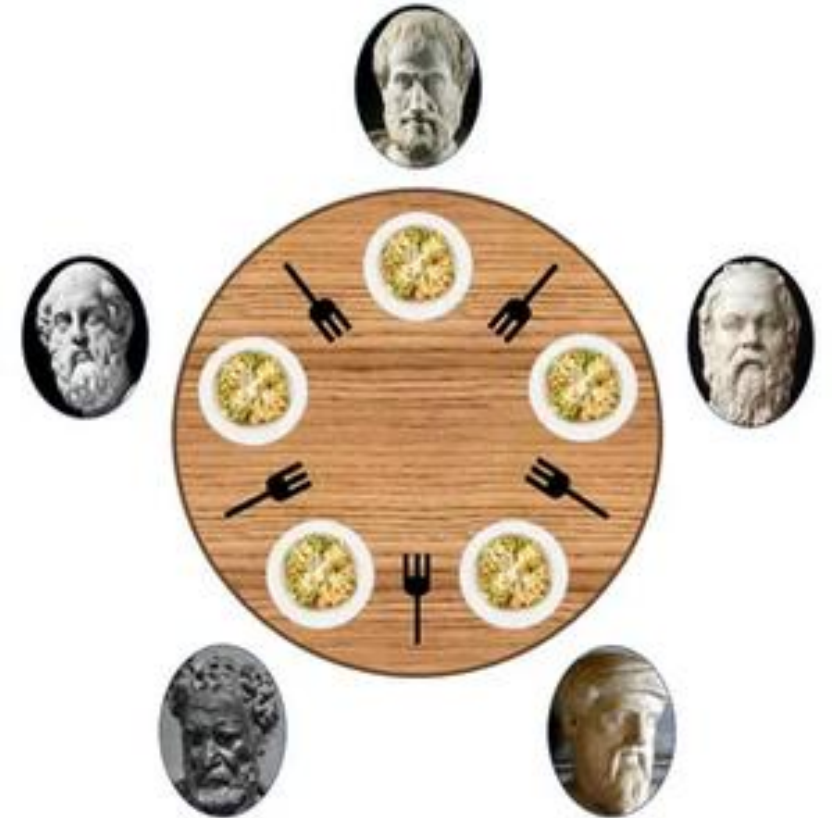


이렇게 동작이 꼬여서 아무도 못 움직이는 상황을
Deadlock, 교착 상태라 부름

Deadlock 의 고전적인 비유

- Deadlock을 설명할 때 가장 널리 쓰이는 비유로 식사하는 철학자 문제라는게 있음
1965년에 다익스트라 라는 아저씨가 만듦
(참고사항으로, 자료구조 12주차에 다익스트라 알고리즘 할건데 그것도 이 아저씨가 만듦)
- 이 문제에서 가정하는 상황은 아래와 같음
 - 다섯 명의 철학자가 원탁에 앉아서 식사를 한다
 - 각 철학자 사이에는 포크가 하나씩 있고, 앞에는 스파게티가 담긴 접시가 있다
 - 메뉴가 스파게티인 관계로 무조건 양손에 포크를 하나씩 쥐고 먹어야만 한다
 - 각 철학자는 식사 또는 생각만 할 수 있고 말은 할 수 없다

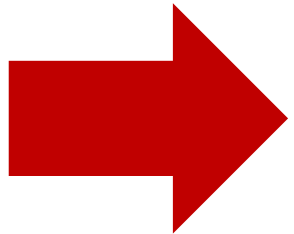
사실 이거 봐도 이해 잘 안될텐데
너무 고전적이고 유명한 비유라서 넣어봄



Dining philosophers problem

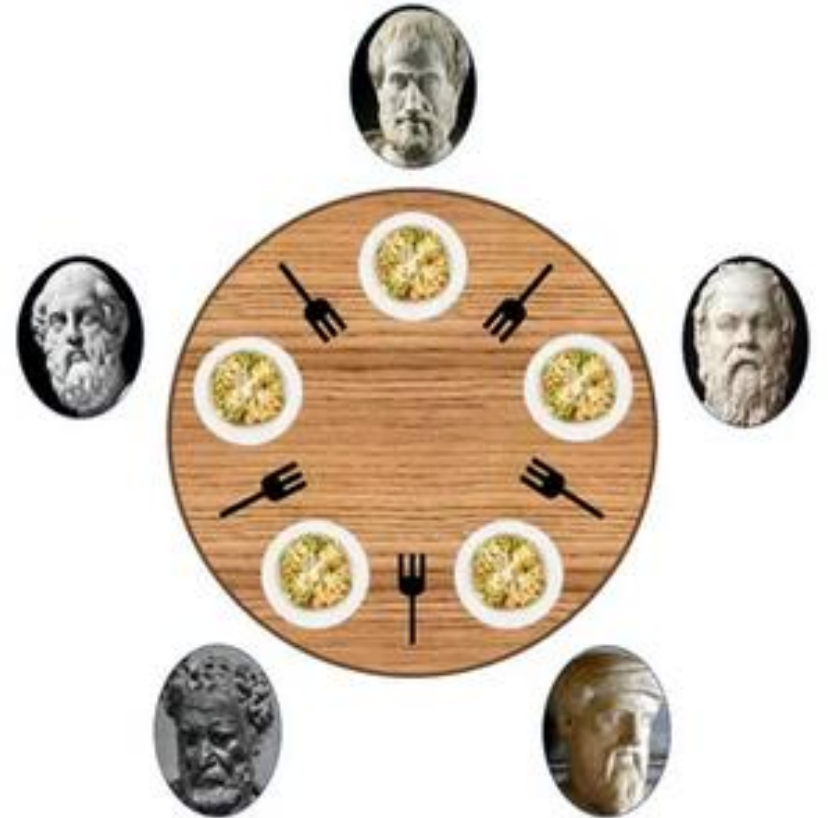
Deadlock 의 고전적인 비유

- 따라서, 철학자가 밥을 먹으려면 왼쪽과 오른쪽의 철학자가 모두 식사를 하지 않는 중이어야만 한다
- 또한, 식사를 마치면 포크를 다른 철학자가 쓸 수 있도록 내려놓는다
- 각 철학자는 서로가 배가 고픈지 아니면 단지 생각하는 중인지 알 수 없다



이 때 어떤 철학자도 굶지 않고
식사할 수 있는 방법은?

사실 이거 봐도 이해 잘 안될텐데
너무 고전적이고 유명한 비유라서 넣어봄



Dining philosophers problem

Deadlock 의 고전적인 비유

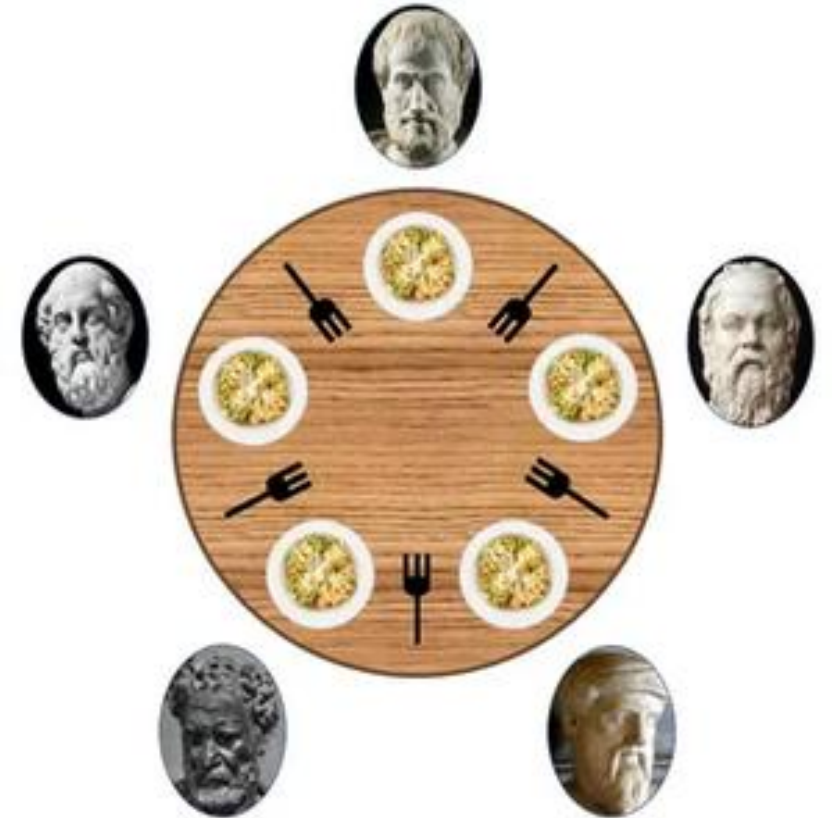
- 일단 각 철학자는 왼쪽 포크부터 집어든 다음, 오른쪽 포크가 사용 가능해질 때까지 대기한다고 하자 (사용 가능하면 집어 들고 식사함)

**모든 철학자가 왼쪽 포크만
집어 들고 식사를 못하게 됨**

- 이번에는 각 철학자가 양 쪽의 포크를 한번에 집어 든다 하자
이렇게 하면 일단 식사는 가능할거같음

**근데 일부 철학자는 밥을 못먹고
무한정 굶게 됨 (Starvation)**

사실 이거 봐도 이해 잘 안될텐데
너무 고전적이고 유명한 비유라서 넣어봄



Dining philosophers problem

Deadlock의 정의

⚠ Deadlock 이란?

둘 이상의 실행 흐름(Thread/Process)이
서로가 보유한 **자원**을 기다리며 영원히 진행하지 못하는 상태



여러 실행 흐름

최소 2개 이상의 스레드/프로세스 존재



여러 자원 점유

각자 자원을 하나 이상 보유함



상호 순환 의존

상대방 자원을 교차 요구함

Resource

컴퓨터 시스템에서 Thread/Process가 작업을 수행하기 위해 필요한 모든 한정된 요소



소프트웨어 자원

- Mutex / Semaphore
- Database Lock
- File Lock



하드웨어 자원

- Printer / Scanner
- GPU 메모리 Buffer
- Network Socket



시스템 자원

- Shared Memory Block
- DB Connection Pool
- Process ID Table

02

교착 상태의 발생 조건

Deadlock의 발생 조건

Coffman Conditions

Mutual Exclusion

Hold and Wait

No Preemption
(비선점)

Circular Wait

Coffman Condition

위 4개의 조건이 동시에 만족되면
Deadlock 발생 가능함

02 교착 상태의 발생 조건 Deadlock의 발생 조건

1번 조건: Mutual Exclusion

개념 정의

"한 번에 한 프로세스만 자원을 사용할 수 있다"

자원이 독점적이어서 다른 프로세스가 사용 중이라면
무조건 기다려야 하는 성질



예시: 프린터 또는 Mutex Lock

Thread A가 프린터로 출력 중일 때,
Thread B는 출력이 끝날 때까지 **프린터에 접근 불가**

누구나 동시 접근 가능한 자원(Read-only 파일 등)은
Deadlock 안생김

02 교착 상태의 발생 조건 Deadlock의 발생 조건

2번 조건: Hold and Wait

개념 정의

"자원을 손에 쥐고 있으면서, 다른 자원을 원함"

최소한 하나 이상의 자원을 보유(Hold)한 채로,
다른 프로세스가 사용 중인 자원을 추가로 요구하며 대기(Wait)

Thread A 상태 도식화:

Thread A
└─ Holds ─→ [Resource 1]
└─ Waits ─→ [Resource 2] (held by B)

3번 조건: No Preemption

개념 정의

"남이 가진 자원을 강제로 빼앗을 수 없다"

다른 프로세스가 보유한 자원을 강제로 빼앗아 올 수 없으며,
보유자가 스스로 해제(Release)할 때까지 기다려야 함

⚠ 주의: CPU Preemption과의 차이

4주차에 배운 CPU Preemption은 OS가 CPU 선점권을
강제로 빼앗는 것**이지만**, 여기서 말하는 No Preemption은
프로세스가 획득한 Lock/자원 독점권을 강제로 취소할 수 없다는 의미

4번 조건: Circular Wait

개념 정의

"대기 관계가 고리(Circle) 형태를 이룬다"

P0는 P1의 자원을, P1은 P2의 자원을,
Pn은 다시 P0의 자원을 기다리는 순환 고리가 형성

Thread A → Lock B 필요



Lock A 필요 ← Thread B

🔒 닫힌 고리(Closed Loop) 형성

02 교착 상태의 발생 조건 Deadlock의 발생 조건

Coffman Condition

1. Mutual Exclusion (상호 배제)

자원은 한 번에 한 프로세스만 사용 가능

2. Hold and Wait (점유와 대기)

자원을 쥔 상태에서 다른 자원을 추가로 요구하며 대기

3. No Preemption (비선점)

다른 프로세스의 자원을 강제로 빼앗을 수 없음

4. Circular Wait (환형 대기)

프로세스간 자원 대기 관계가 원형 고리를 형성

위 4개의 조건이 동시에 만족되면
Deadlock 발생 가능함

(하나라도 깨지면 방지 가능)

03

교착 상태의 대응 전략

Deadlock의 대응 전략

03 교착 상태의 대응 전략 Deadlock의 대응 전략

1번 전략: Prevention (예방)

핵심 아이디어: Coffman 4가지 조건 중 **최소 하나**를 사전에 원천 차단

Mutual Exclusion 파괴

모든 자원을 공유 가능하게 만듦 (현실적으로 프린터/Lock 등 독점 자원은 불가능)

Hold and Wait 파괴

실행 전에 필요한 모든 자원을 한번에 요청 및 할당받게 함 (자원 낭비 심함)

No Preemption 파괴

다른 자원이 없으면 쥐고 있던 자원을 빼앗기고 처음부터 다시 시작 (상태 복구 어려움)

Circular Wait 파괴 (실무에서는 이거 제일 자주 씬)

자원에 우선순위 번호를 매기고 정해진 순서대로만 요청하도록 함

Lock Ordering

전역 순서(Global Order) 규칙

"모든 스레드는 자원 획득 시 반드시 미리 정해진 순서(예: Lock A → Lock B)로만 획득해야 한다."

효과: Circular Wait가 절대 발생할 수 없음!

Thread B도 Lock A를 먼저 얻어야 하므로, A가 쥐고 있으면 B는 시작조차 안 함.

✗ BAD (교착 발생 가능):

Thread A: Lock_1 → Lock_2
Thread B: Lock_2 → Lock_1

Lock Ordering 적용:

Thread A: Lock_1 → Lock_2
Thread B: Lock_1 → Lock_2

Prevention의 한계

↓ 자원 활용률 저하

필요하지도 않은 자원을 미리 다 확보해두거나(Hold and Wait 방지 시),
순서를 기다리느라 놓고 있는 자원이 많아짐

🕒 시스템 처리량(Throughput) 감소

동시성(Concurrency)이 과도하게 제한되어
시스템 전체의 작업 속도가 느려짐

문을 다 걸어잠그면 보안은 좋지만
사용하기 불편한거랑 같음

2번 전략: Avoidance

핵심 아이디어: 자원 요청 시 동적으로 상태를 검사하여, **Safe State(안전 상태)**를 유지할 때만 할당한다!

🛡️ Safe State (안전 상태)

모든 프로세스가 무사히 종료될 수 있는 ****Safe Sequence(안전 순서열)****가 최소 하나 이상 존재하는 상태.

→ **Deadlock 절대 안 생김!**

⚠️ Unsafe State (불안전 상태)

자원 요구량에 따라 Deadlock으로 진입할 가능성이 있는 상태.

→ **Deadlock 가능성 존재!**

Banker's Algorithm



잔고: 100만원



잔고: 30만원



A

저는 최대 70만원까지 필요해요
그리고 현재 40만원 빌렸어요
→ 30만원 더 필요해요



B

저는 최대 60만원까지 필요해요
그리고 현재 30만원 빌렸어요
→ 30만원 더 필요해요

Banker's Algorithm

현재 잔고로 A에게 빌려준다면?

1. A가 일을 끝내고 70만원을 갚음
2. 그 70만원으로 B에게 30만원을 빌려줌

→ A에게 30만원 빌려줘도 될듯

Safe State

잔고: 30만원



A: 30만원 필요



B: 30만원 필요

Banker's Algorithm



잔고: 100만원



잔고: 10만원



A

저는 최대 80만원까지 필요해요
그리고 현재 50만원 빌렸어요
→ 30만원 더 필요해요



B

저는 최대 70만원까지 필요해요
그리고 현재 40만원 빌렸어요
→ 30만원 더 필요해요

Banker's Algorithm

잔고: 10만원



현재 잔고로 A에게 빌려준다면?

내가 가진 돈이 부족한데?

A가 돈을 갚을 수 없는데?

A랑 B는 계속 대기할 수밖에 없는데?

Unsafe State

Deadlock



A: 30만원 필요



B: 30만원 필요

Banker's Algorithm

프로그램	최대 필요량(Max)	현재 받은 양(Allocation)	앞으로 필요한 양(Need)
A	7	5	2
B	5	2	3

Banker's Algorithm

컴퓨터에 자원이 9개 있다면?

A → B 순서로 가면 모두 끝낼 수 있을듯

Banker's Algorithm

Banker's Algorithm

1. 현재 남은 자원이 몇 개인가?
2. 그 남은 자원만으로 끝낼 수 있는 프로그램은?
3. 있으면 그 프로그램이 쓰던 자원 돌려받음
4. 이렇게 모든 프로그램 끝낼 수 있으면 Safe State
5. 끝낼 수 없으면 Unsafe State, Deadlock 발생 가능성 높음

위험해질 것 같으면 애초에 자원을 주지 말자



Avoidance

Avoidance의 한계

미래 요구량 예측 불가

프로세스가 실행 중 최대 얼마나 자원을 쓸지
미리 정확히 선언하기 어려움



동적 프로세스 변화

현대 OS에서는 프로세스 수가 끊임없이
변하므로 고정된 자원 계산이 불가



연산 오버헤드

자원 할당 요청이 올 때마다
안전성 검사 알고리즘을 수행하면 성능이 떨어짐

3번 전략: Detection & Recovery

🔍 1. Deadlock Detection

발생을 막지 않고 일단 그냥 둬

주기적으로 Deadlock 탐지 알고리즘을 실행해
Deadlock 상태 발생 여부 감지

🚑 2. Deadlock Recovery

- Process Termination (강제 종료):
 - 교착 상태의 모든 프로세스 종료
 - 또는 하나씩 강제 종료하며 교착 풀리는지 확인
- Resource Preemption (자원 선점):
 - Victim 프로세스의 자원을 강제 몰수하여
이전 상태로 롤백

04

기타

Miscellaneous

Ostrich Algorithm

- 이걸 알고리즘이라고 불러야 되나 싶긴 한데 암튼
- Deadlock이 발생하는 빈도가 매우 드물수 있음
- 그럼 Deadlock이 실제로 발생했을 때
그냥 재부팅 또는 해당 프로그램만 꺼버림
- 그게 예방, 회피, 감지를 계속 수행하는
비용보다 훨씬 경제적이었음
- 현대의 Windows, Linux, macOS가
이 방법을 채택함



Starvation & Livelock

구분	Deadlock (교착)	Starvation (기아)	Livelock (라이브락)
상태 (State)	Blocked (대기 상태)	Ready / Waiting	Running (진행 중)
CPU 점유	0% (CPU 안 씀)	0% (스케줄링 못 받음)	100% (CPU 계속 씀)
원인	상호 순환 자원 대기	우선순위에서 계속 밀림	서로 양보만 무한 반복
비유	좁은 길 마주쳐 멈춤	새치기당해 줄 못 서움	복도에서 비켜주다 부딪힘

Starvation: 다른 애들한테 계속 밀려서 내 차례가 안옴

Livelock: 복도에서 두 사람이 마주쳤는데
서로 계속 왼쪽 오른쪽으로 비킴
(두 사람 모두 계속 움직이지만 아무도 앞으로 못감)